

Blocco 3 – PostgreSQL in Docker

Guida operativa per testare creazione tabelle e query

Obiettivo

Nel blocco 3 si ragiona su normalizzazione, granularità, dipendenze funzionali e schema relazionale. Per non lasciare questi concetti solo su carta, questa guida mostra come aprire un database PostgreSQL temporaneo e usarlo per verificare concretamente uno schema normalizzato.

L'idea è semplice:

1. avviare PostgreSQL in Docker;
2. entrare nel database con `psql`;
3. creare uno schema di test;
4. caricare tabelle normalizzate;
5. inserire dati coerenti;
6. eseguire query per ricostruire il report iniziale e controllare che lo schema funzioni.

Prerequisiti

- Docker installato e avviato.
- Terminale disponibile.
- Materiale del corso scaricato localmente.
- Conoscenza minima della shell: copiare ed eseguire un comando.

Non serve installare PostgreSQL sul computer: il DBMS gira dentro il container.

1. Avviare PostgreSQL

Dalla cartella principale della repository, eseguire:

```
docker compose up -d postgres
```

Il file `docker-compose.yml` definisce:

- container `rdsq1-postgres`;
- immagine `postgres:16`;
- utente `training`;
- password `training`;
- database `training`;
- porta `5432` esposta sul computer.

Se la porta `5432` è già occupata:

```
ports:  
  - "15432:5432"
```

In questo caso modificare `docker-compose.yml` prima di avviare il container. Dentro il container PostgreSQL continuerà a usare la porta 5432.

2. Controllare che il container sia attivo

```
docker compose ps
```

Per leggere i messaggi di avvio:

```
docker logs rdsq1-postgres
```

Un messaggio simile a `database system is ready to accept connections` indica che PostgreSQL è pronto.

3. Entrare in PostgreSQL con psql

```
docker exec -it rdsq1-postgres psql -U training -d training
```

Da questo momento si è dentro la console SQL di PostgreSQL. Il prompt sarà simile a:

```
training=#
```

Comandi utili di psql:

- `\conninfo`: mostra connessione corrente;
- `\l`: elenca i database;
- `\dn`: elenca gli schemi;
- `\dt`: elenca le tabelle dello schema corrente;
- `\d nome_tabella`: mostra colonne e vincoli di una tabella;
- `\q`: esce da psql.

Questi comandi iniziano con `\` e non sono SQL standard: sono comandi della console psql.

4. Primo test manuale

Dentro psql, eseguire:

```
CREATE SCHEMA block03_lab;  
SET search_path TO block03_lab;  
  
CREATE TABLE customers (  
    customer_id INTEGER PRIMARY KEY,  
    customer_name TEXT NOT NULL,  
    city TEXT NOT NULL  
);  
  
INSERT INTO customers (customer_id, customer_name, city) VALUES  
    (1, 'Alfa Market', 'Roma'),  
    (2, 'Beta Shop', 'Milano');  
  
SELECT * FROM customers;
```

Verifica:

- `CREATE SCHEMA` crea un contenitore logico per le tabelle del blocco 3;
- `SET search_path` dice a PostgreSQL di cercare le tabelle in quello schema;
- `PRIMARY KEY` impedisce due clienti con lo stesso identificativo;
- `NOT NULL` impedisce valori mancanti dove il dato è obbligatorio;
- `SELECT *` permette di controllare i dati inseriti.

Per eliminare il test:

```
DROP SCHEMA block03_lab CASCADE;
```

`CASCADE` elimina anche le tabelle contenute nello schema.

5. Caricare lo script completo del blocco 3

Dalla cartella principale del corso, eseguire:

```
docker exec -i rdsq1-postgres psql -U training -d training \  
< activities/block03_normalizzazione_schema/postgresql_docker_test.sql
```

Lo script:

- crea lo schema `block03_lab`;
- crea tabelle normalizzate per clienti, prodotti, vendite e righe vendita;
- inserisce dati di esempio;
- esegue query di controllo.

Se si preferisce entrare prima in `psql`, si può copiare il contenuto del file e incollarlo nella console.

6. Interagire con lo schema caricato

Entrare nel database:

```
docker exec -it rdsq1-postgres psql -U training -d training
```

Selezionare lo schema:

```
SET search_path TO block03_lab;
```

Elencare le tabelle:

```
\dt
```

Controllare la struttura di una tabella:

```
\d sales
```

Leggere i dati:

```
SELECT * FROM customers;  
SELECT * FROM products;  
SELECT * FROM sales;  
SELECT * FROM sale_items;
```

7. Query di controllo

Ricostruire il report iniziale

Uno schema normalizzato distribuisce l'informazione in più tabelle. Per verificare che non abbiamo perso contenuto informativo, dobbiamo poter ricostruire una vista simile alla tabella iniziale.

```
SELECT
  s.sale_id,
  s.sale_date,
  c.customer_name,
  c.city,
  p.sku,
  p.product_name,
  si.quantity,
  si.unit_price
FROM sales AS s
JOIN customers AS c
  ON c.customer_id = s.customer_id
JOIN sale_items AS si
  ON si.sale_id = s.sale_id
JOIN products AS p
  ON p.product_id = si.product_id
ORDER BY s.sale_id, si.line_no;
```

La query anticipa il tema dei JOIN, che sarà ripreso nei blocchi successivi. Qui serve come test pratico: se il report si ricostruisce, le tabelle sono collegate correttamente.

Calcolare il totale di ogni vendita

```
SELECT
  s.sale_id,
  s.sale_date,
  c.customer_name,
  SUM(si.quantity * si.unit_price) AS sale_total
FROM sales AS s
JOIN customers AS c
  ON c.customer_id = s.customer_id
JOIN sale_items AS si
  ON si.sale_id = s.sale_id
GROUP BY s.sale_id, s.sale_date, c.customer_name
ORDER BY s.sale_id;
```

Questa query controlla che quantità e prezzo siano nel posto giusto: non descrivono il prodotto in generale, ma la riga specifica di una vendita.

Cercare dati non rappresentabili nella tabella unica

```
SELECT
  product_id,
  sku,
  product_name
FROM products
WHERE product_id NOT IN (
  SELECT product_id
  FROM sale_items
);
```

Questa query mostra un vantaggio della separazione: posso registrare un prodotto anche se non è ancora stato venduto.

8. Errori utili da mostrare in aula

Provare a inserire due clienti con la stessa chiave:

```
INSERT INTO customers (customer_id, customer_name, city)
VALUES (1, 'Cliente duplicato', 'Torino');
```

PostgreSQL rifiuta l'inserimento perché `customer_id` è chiave primaria.

Provare a inserire una vendita per un cliente inesistente:

```
INSERT INTO sales (sale_id, sale_date, customer_id)
VALUES (999, DATE '2026-05-07', 999);
```

PostgreSQL rifiuta l'inserimento perché `customer_id = 999` non esiste in `customers`. Questo è il vincolo di chiave esterna.

9. Resettare il laboratorio

Per cancellare e ricreare lo schema di test:

```
docker exec -i rdsq1-postgres psql -U training -d training \
< activities/block03_normalizzazione_schema/postgresql_docker_test.sql
```

Lo script contiene all'inizio:

```
DROP SCHEMA IF EXISTS block03_lab CASCADE;
```

Quindi ogni esecuzione riparte da uno stato pulito.

10. Fermare, riavviare o rimuovere il container

Fermare PostgreSQL:

```
docker compose stop postgres
```

Riavviarlo:

```
docker compose up -d postgres
```

Rimuovere completamente container e dati del laboratorio:

```
docker compose down -v
```

La rimozione elimina anche il volume Docker con i dati del laboratorio.

11. Problemi comuni

Il nome del container esiste già. Se Docker segnala che il nome `rdsq1-postgres` è già in uso:

```
docker compose up -d postgres
```

oppure, se si vuole ripartire da zero:

```
docker compose down -v
docker compose up -d postgres
```

La porta 5432 è occupata. Modificare `docker-compose.yml` e usare una porta locale alternativa:

```
ports:
  - "15432:5432"
```

Quando si usa `docker exec`, la porta host non conta perché ci si collega dall'interno del container. **relation does not exist.** Probabilmente lo schema corrente non è `block03_lab`. Eseguire:

```
SET search_path TO block03_lab;
\dt
```

password authentication failed. Controllare di usare:

- utente: `training`;
- password: `training`;
- database: `training`.

12. Sequenza consigliata in aula

- 10 minuti: avvio container e accesso a `psql`;
- 10 minuti: comandi `psql` essenziali;
- 20 minuti: creazione manuale di una tabella semplice;
- 20 minuti: caricamento dello script completo;
- 20 minuti: query di verifica e discussione sul perché lo schema normalizzato ricostruisce il report iniziale;
- 10 minuti: errori guidati su chiave primaria e chiave esterna.